

## 2.3.3 Number of Repetitions

دہرائی (لوپ) کارکردگی کا سب سے بڑا عامل ہے۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 11 of 24

### 2.3.3 Number of Repetitions

#### LEARN IT

**Repetitions** (loops) are the most significant factor in algorithm efficiency. The **structure of the loop** decides how time grows with  $n$ .

| Type of repetition     | Conceptual impact                                                                   | Efficiency grade        |
|------------------------|-------------------------------------------------------------------------------------|-------------------------|
| Linear loop            | Processes each element exactly once. Time grows in proportion to size.              | Good — $O(n)$           |
| Logarithmic repetition | Reduces the problem size by half each iteration (binary search). Huge time savings. | Excellent — $O(\log n)$ |
| Nested loops           | For each element, process all other elements. Operations explode.                   | Poor — $O(n^2)$         |

#### WORKED EXAMPLE

**Worked example: count the inner-body runs.**

```
n = 5
linear = 0
for i in range(n):
    linear = linear + 1

nested = 0
for i in range(n):
    for j in range(n):
        nested = nested + 1

print(linear, nested)
```

#### OUTPUT

5 25

One loop: 5. Nested loops:  $25 = 5^2$ . For  $n = 1,000$  that would be 1,000 versus 1,000,000.

#### REMEMBER IT

Find the **deepest loop**. One loop  $\approx O(n)$ . Loop inside a loop  $\approx O(n^2)$ . A loop that halves  $n \approx O(\log n)$ .

#### COMMON MISTAKE

**Mistake:** counting the word `for` and ignoring how the index moves.

**Fix:** a loop `while n > 1: n = n // 2` is logarithmic, even though it is still “a loop”.

اردو میں سمجھیں

دہرائی (لوپ) کارکردگی کا سب سے بڑا عامل ہے۔ ایک بار ہر عنصر پر چلنے والا لوپ  $O(n)$  (اچھا)، ہر بار مسئلہ آدھا کرنے والا  $O(\log n)$  (بہترین)، اور اندرونی لوپ  $O(n^2)$  (خراب) ہے۔ اندرونی جسم کے چلنے کی گنتی سے یہ فرق صاف نظر آتا ہے۔

#### CHECK YOURSELF

1. What grade does the source give nested loops?
2. Why is binary search's loop logarithmic rather than linear?
3. If  $n = 5$ , how many times does a fully nested  $n$ -by- $n$  pair of loops run?

Answers: 1. Poor —  $O(n^2)$ . 2. Each iteration halves the remaining problem, so the number of iterations is about  $\log n$ . 3. 25.

---

Next lecture: 2.3.4 The Efficiency Evaluation Framework